Table of Contents

Introducción a Django
- Ventajas de Django
- Ciclo de vida de las versiones

Configuración inicial en PyCharm
- Crear un proyecto Django
- Funcionamiento de una aplicación web
- Arquitectura de Django
- Estructura del proyecto
- Ejecutar el servidor local
- Diferencia entre proyecto y aplicación

Configuración de la base de datos

Django ORM: Relaciones y Ejemplo Práctico

Introducción al ORM de Django
- Configuración Inicial
- Modelos
- Explicación de los modelos
- Relaciones:
- Migraciones
- Datos de Ejemplo

CRUD Completo
- Crear (Create)
  - Vista para crear empleados
  - Template para crear empleados
  - URLs
- Listado
  - Vista para listar empleados
  - Template para listar empleados
  - URLs
- Detalle
  - Template para ver el detalle de un empleado
  - URLs

Actualizar (Update)
Vista para editar empleados
Template para editar empleados
URLs
Eliminar (Delete)
Vista para eliminar empleados
URLs

Introducción a Django

Django es un framework de aplicaciones web gratuito y de código abierto (open-source) escrito en Python. Este tipo de herramientas permite desarrollar sitios web de manera más rápida y eficiente al reutilizar componentes preexistentes.

Ventajas de Django

- **Código abierto y gratuito.**
- **Versatilidad:** Adecuado para aplicaciones monolíticas y de microservicios.
- **Conjunto de componentes integrados:** Enrutamiento, autenticación, manejo de sesiones, entre otros.
- **Comunidad activa:** Amplia documentación y soporte.
- **Adopción por grandes empresas:** Usado por plataformas como Instagram, Spotify y YouTube.

Ciclo de vida de las versiones

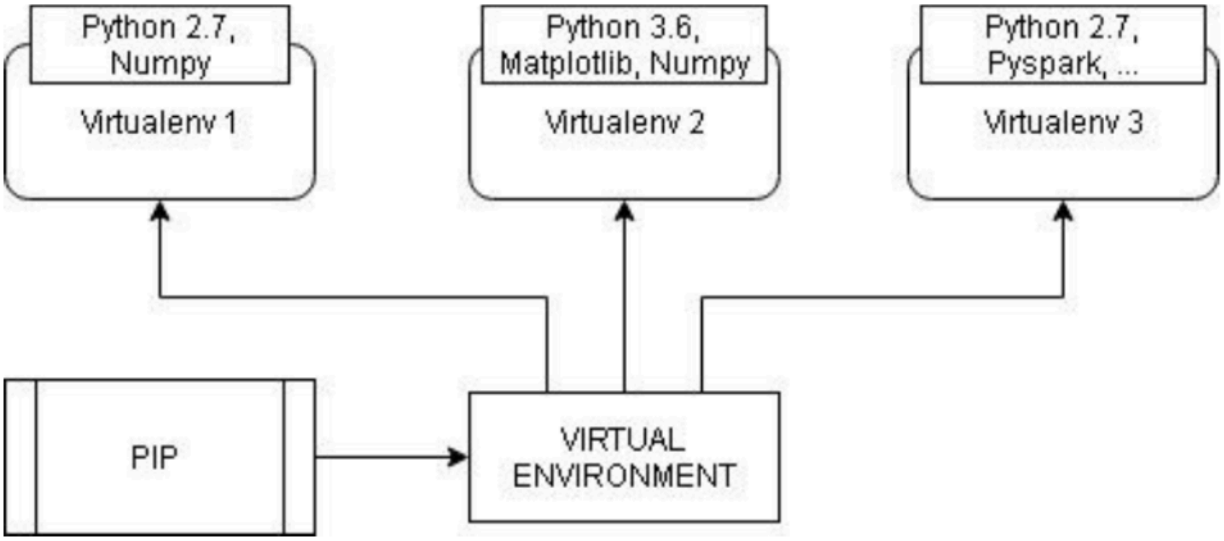
Actualmente, trabajamos con la versión 5 de [Django](Download Django | Django), que tendrá soporte extendido hasta abril de 2028. Las futuras actualizaciones mantendrán este ciclo de soporte.

Configuración inicial en PyCharm

Crear un proyecto Django

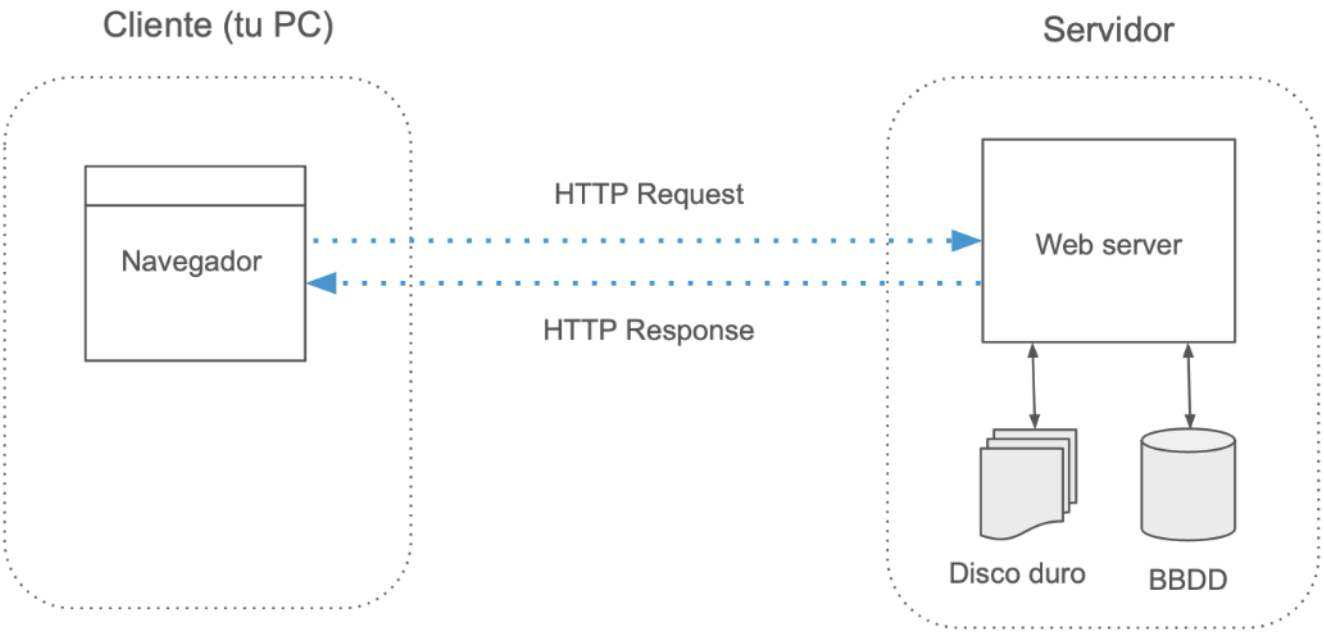
1. Ve a **File** → **New Project**→**Empresa_django_funciones**.
2. Selecciona `Proyect venv` como entorno virtual y elige la última versión de Python disponible en tu entorno.

Un entorno virtual permite tener uno o más espacios aislados donde poder instalar distintas versiones de programas y paquetes, evitando y aislando así conflictos de versiones de librerías, etc.

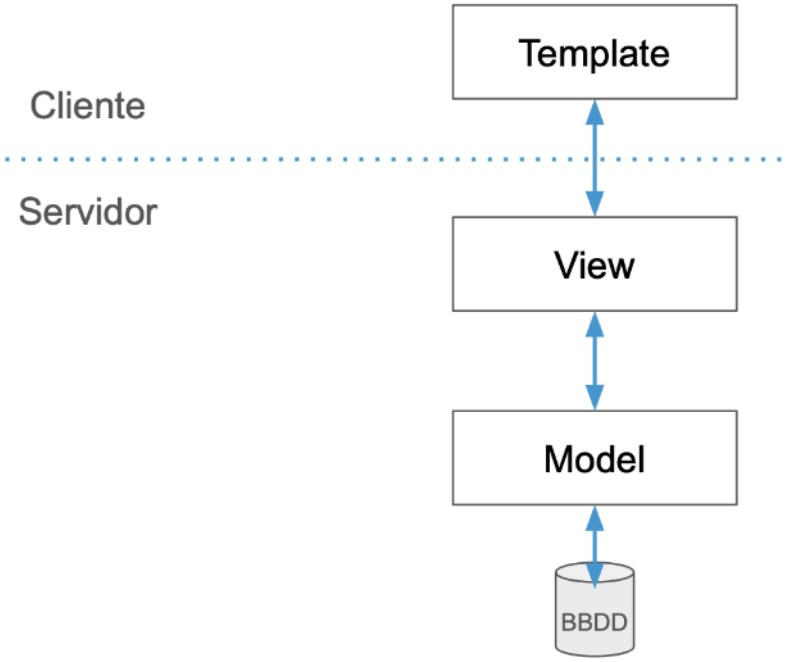


Funcionamiento de una aplicación web

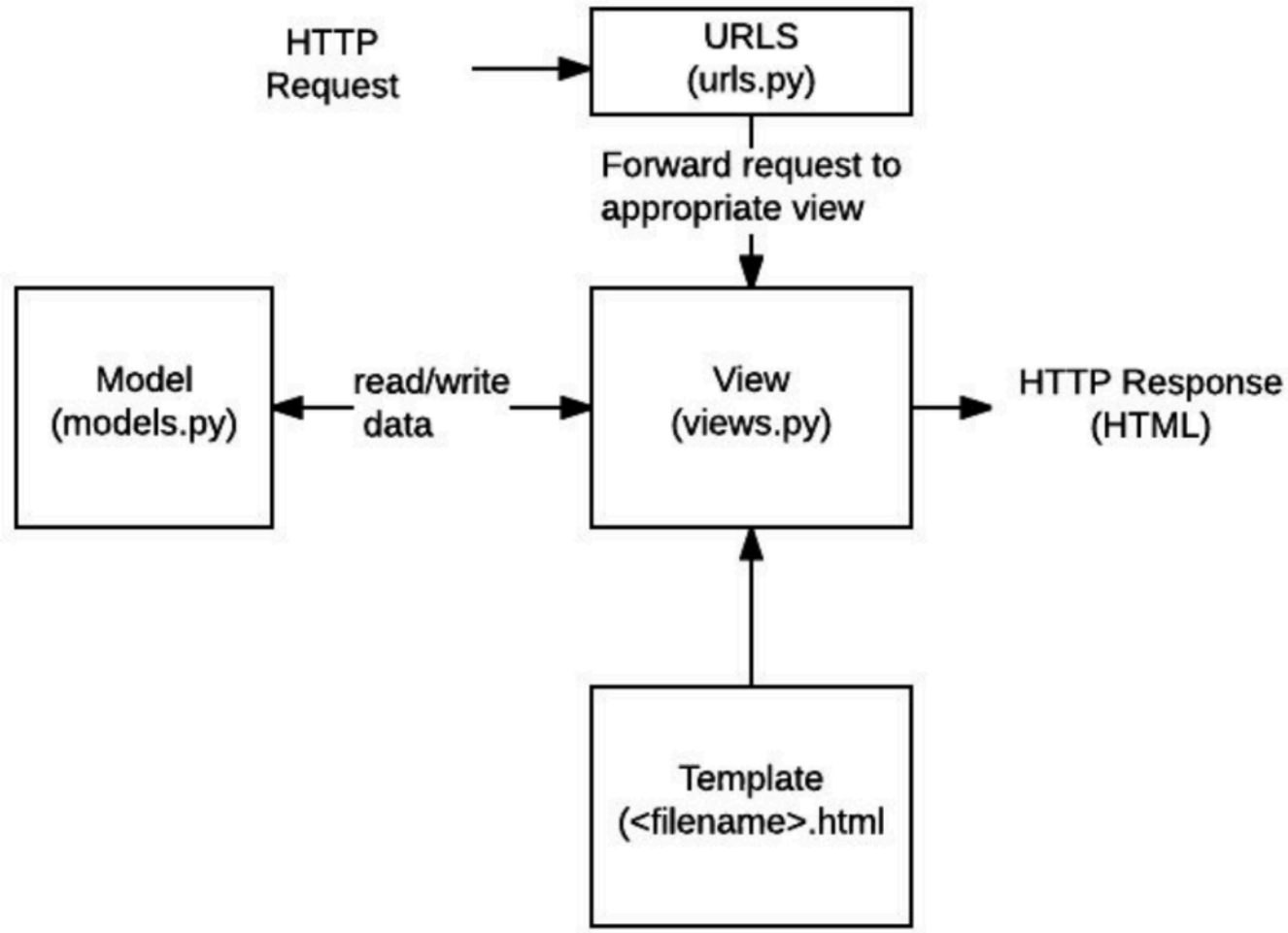
1. La aplicación web espera peticiones HTTP del cliente (normalmente un navegador web).
2. Al recibir una petición la aplicación extrae lo que se necesita basándose en la URL y la información incluida en los datos POST o GET
3. Realiza las operaciones necesarias para satisfacer la petición (por ejemplo, leer/escribir en la base de datos)
4. La aplicación devuelve una respuesta al cliente (normalmente creando dinámicamente una página HTML para que el navegador la interprete y la muestre)



Arquitectura de Django



Se traduce en:



- **URLs**: El mapeador URL se usa para redirigir las peticiones HTTP a la vista apropiada basándose en la URL de la petición.
- **Vista (View)**: gestiona las peticiones HTTP que recibe y devuelve respuestas HTTP. Las vistas acceden a los datos que necesitan para satisfacer las peticiones por medio de modelos, y delegan el formateo de la respuesta a las plantillas ("templates").
- **Modelos (Models)**: son objetos que definen la estructura de los datos de una aplicación y proporcionan mecanismos para gestionar (añadir, modificar y borrar) y consultar registros en la base de datos.
- **Plantillas (Templates)**: una plantilla es un fichero de texto que define una estructura de otro fichero. Una vista puede crear dinámicamente una página usando una plantilla, rellenandola con datos de un modelo.

Estructura del proyecto

- **init.py**: archivo vacío para que Python reconozca la carpeta como un paquete y te permita usar sus objetos dentro de otras partes del proyecto.
- **manage.py:** Script para gestionar el proyecto desde la terminal.
- **settings.py:** Configuración global del proyecto.
- **urls.py:** Manejo de rutas (puntos de entrada) a la aplicación y su enrutamiento.
- **wsgi.py:** Configuración para el despliegue en producción.

Ejecutar el servidor local

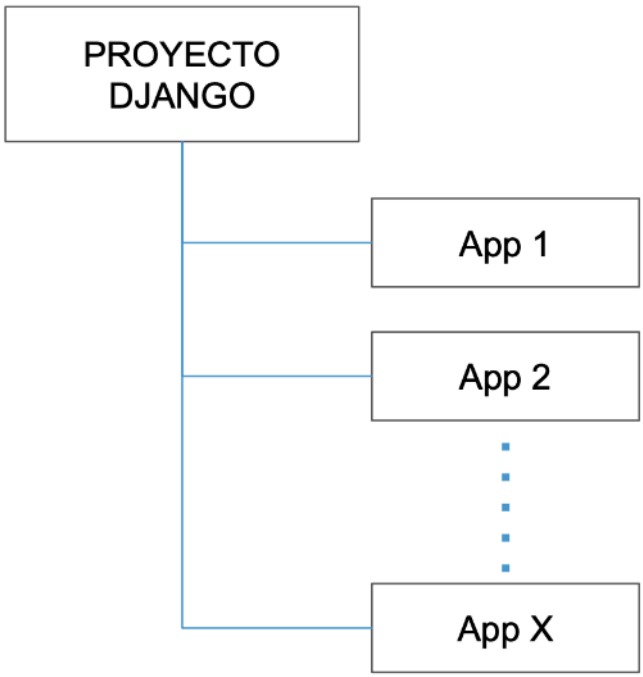
```
python manage.py runserver
```

Comprobar que se accede correctamente desde http://127.0.0.1:8000.

Nota: se trata de un servidor ligero para pruebas, no se recomienda utilizarlo en entorno de producción.

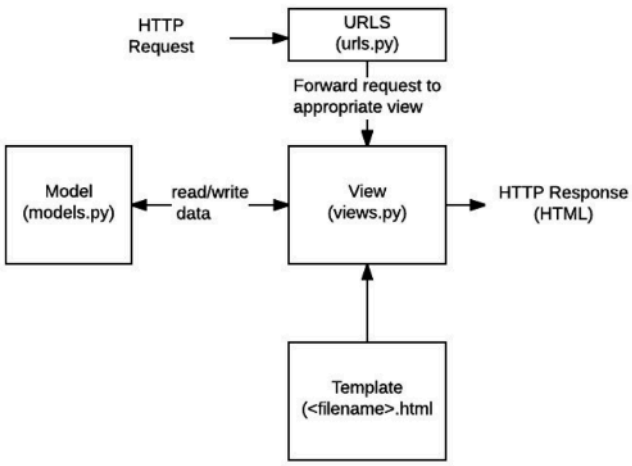
Diferencia entre proyecto y aplicación

Un proyecto Django puede contener varias aplicaciones. Las aplicaciones son componentes reutilizables que encapsulan funcionalidades específicas.



Más adelante crearemos una aplicación y la añadiremos en la lista de aplicaciones de nuestro proyecto.

```
myproject/
    manage.py
    myproject/
    myapp/
        admin.py
        apps.py
        models.py
        tests.py
        views.py
        __init__.py
        migrations/
```



Por defecto, con la propia instalación si examinamos el fichero `settings.py` veréis como ya hay ciertas aplicaciones disponibles en nuestro proyecto:

```
31  # Application definition
32
33  INSTALLED_APPS = [
34      'django.contrib.admin',
35      'django.contrib.auth',
36      'django.contrib.contenttypes',
37      'django.contrib.sessions',
38      'django.contrib.messages',
39      'django.contrib.staticfiles',
40  ]
```

Una vez creada la aplicación, tienes que añadirla al listado de aplicaciones en settings.py del proyecto:

```
INSTALLED_APPS = [
    'django.contrib.admin' ,
    'django.contrib.auth' ,
    'django.contrib.contenttypes' ,
    'django.contrib.sessions' ,
    'django.contrib.messages' ,
    'django.contrib.staticfiles' ,
    'appEmpresaDjango'
]
```

Además dentro de ese fichero `settings.py`, vamos a encontrar la configuración del proyecto.

- **DEBUG = True**. El modo de DEPURACIÓN, es un modo de ejecución especial en el que siempre que ocurra un fallo, Django nos mostrará un montón de información para analizarla, saber dónde se ha originado el error y poder así solucionarlo. Óbviamente esta información es confidencial, ya que muestra valores de variables, rutas de directorios y otros datos importantes. Es por eso que en cuanto despleguemos un proyecto de Django en real, lo primero que haremos siempre es desactivar el modo **DEBUG** poniéndolo a *False*.

- Podemos elegir el idioma por defecto de la interfaz de django, así como zona horaria:

```
# Internationalization # https://docs.djangoproject.com/en/dev/topics/i18n/
LANGUAGE_CODE = 'es-es'

TIME_ZONE = 'Europe/Madrid'
```

Configuración de la base de datos

Django utiliza bases de datos relacionales y, por defecto, SQLite3. En el fichero **settings.py** y buscaremos el diccionario llamado *DATABASES*:

```
# Database # https://docs.djangoproject.com/en/dev/ref/settings/#databases
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': os.path.join(BASE_DIR, 'db.sqlite3'),
    }
}
```

Lo que tenemos aquí es una configuración predeterminada que utiliza la base de datos SQLite3. Django trabaja con bases de datos SQL relacionales, y es compatible con los siguientes [engines](Databases | Django documentation | Django):

- PostgreSQL
- MariaDB
- MySQL
- Oracle
- SQLite

Para inicializar la base de datos:

```
python manage.py migrate
```

Django ORM: Relaciones y Ejemplo Práctico

Introducción al ORM de Django

El ORM (Object-Relational Mapping) de Django permite interactuar con la base de datos utilizando objetos de Python en lugar de consultas SQL. Esto simplifica el desarrollo y el mantenimiento del código.

En este documento, trabajaremos con un ejemplo práctico de tres modelos relacionados: **Empleado**, **Departamento** y **Habilidad**. Además, implementaremos un CRUD completo para gestionar los datos de empleados.

Configuración Inicial

1. Crear una aplicación:

   ```
   python manage.py startapp empresa
   ```

   Esto creará una estructura de directorios donde se definirán los modelos, vistas, templates y archivos relacionados con la aplicación.

2. Registrar la aplicación en `INSTALLED_APPS` dentro de `settings.py`:

   ```
   INSTALLED_APPS = [
       ...
       'empresa',
   ]
   ```

   Esto asegura que Django reconozca la aplicación y pueda gestionar sus modelos y configuraciones.

Modelos

Definimos los modelos en `empresa/models.py` para representar las tablas de la base de datos.

```python
from django.db import models

class Departamento(models.Model):
    # No es necesario crear un campo para la Primary Key, Django creará
automáticamente un IntegerField.
    nombre = models.CharField(max_length=50)
    telefono = models.IntegerField()
    created = models.DateTimeField(auto_now_add=True,
                                   verbose_name="Fecha de creación")
    updated = models.DateTimeField(auto_now=True,
                                   verbose_name="Fecha de edición")
    def __str__(self):
        return f"{self.nombre} ({self.telefono})"

class Habilidad(models.Model):
    # No es necesario crear un campo para la Primary Key, Django creará
automáticamente un IntegerField.
    nombre = models.CharField(max_length=50)
    def __str__(self):
        return f"{self.nombre}"
    created = models.DateTimeField(auto_now_add=True,
                                   verbose_name="Fecha de creación")
    updated = models.DateTimeField(auto_now=True,
                                   verbose_name="Fecha de edición")

class Empleado(models.Model):
    # Campo para la relación one-to-many (un empleado pertenece a un
departamento)
    departamento = models.ForeignKey(Departamento, on_delete=models.CASCADE)
    # Campo para la relación many-to-many (un empleado tiene varias
habilidades)
    habilidades = models.ManyToManyField(Habilidad)
    nombre = models.CharField(max_length=40)
    fecha_nacimiento = models.DateField()
    # Es posible indicar un valor por defecto mediante 'default'
    antiguedad = models.IntegerField(default=0)
    created = models.DateTimeField(auto_now_add=True,
                                   verbose_name="Fecha de creación")
    updated = models.DateTimeField(auto_now=True,
                                   verbose_name="Fecha de edición")
    # Para permitir propiedades con valor null, añadiremos las opciones
null=True, blank=True.
    def __str__(self):
        return f"{self.nombre} --- {self.antiguedad}"
```

Explicación de los modelos

- **Departamento**: Representa un área de la empresa con los campos:
  - `nombre`: Nombre del departamento.
  - `telefono`: Número de contacto.
  - `created` y `updated`: Fechas de creación y actualización automáticas.
- **Habilidad**: Representa una habilidad que puede tener un empleado, con:
  - `nombre`: Nombre de la habilidad.
  - `created` y `updated`: Fechas de creación y actualización automáticas.
- **Empleado**: Representa a una persona que trabaja en la empresa, con:
  - `nombre`: Nombre del empleado.
  - `fecha_nacimiento`: Fecha de nacimiento.
  - `antiguedad`: Años de antigüedad en la empresa.
  - `departamento`: Relación de uno a muchos con `Departamento`.
  - `habilidades`: Relación de muchos a muchos con `Habilidad`.
  - `created` y `updated`: Fechas de creación y actualización automáticas.

Relaciones:

- **Empleado** y **Departamento**: Relación de uno a muchos (ForeignKey).
- **Empleado** y **Habilidad**: Relación de muchos a muchos (ManyToManyField).

Migraciones

1. Crear las migraciones:

   ```
   python manage.py makemigrations
   ```

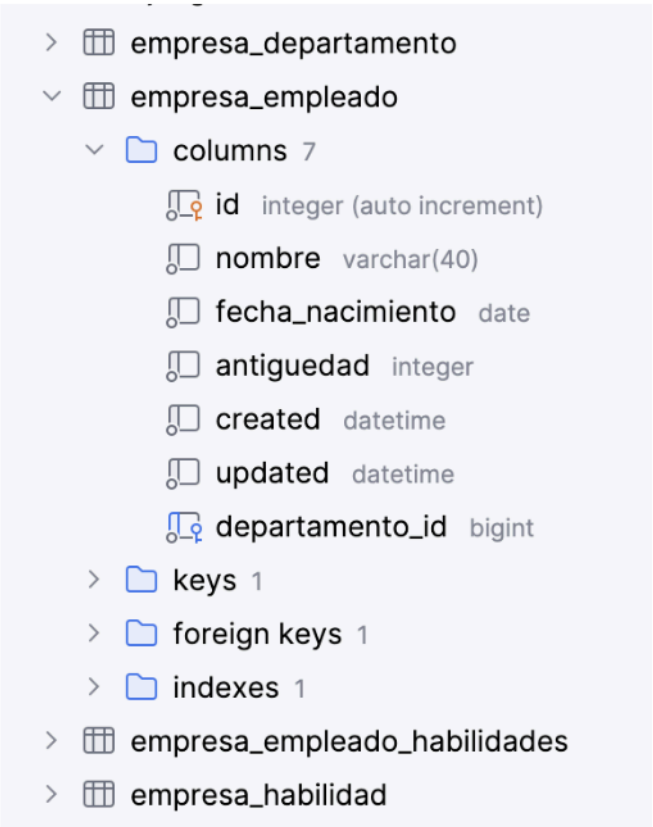
   Esto genera los archivos de migración basados en los modelos definidos.

2. Aplicar las migraciones:

   ```
   python manage.py migrate
   ```

   Esto crea las tablas correspondientes en la base de datos.

3. Compruébalo desde Pycharm añadiendo una conexión SQLite3 haciendo referencia al fichero db.sqlite3 que hay en la raíz del proyecto. Entre otras tablas aparecerán las que hemos definido en nuestro modelo:



Datos de Ejemplo

Para crear datos iniciales, puedes usar el shell de Django desde el terminal que va integrado en Pycharm:

```
python manage.py shell
```

```python
from empresa.models import Departamento, Habilidad, Empleado

# Crear departamentos
dep1 = Departamento.objects.create(nombre="Recursos Humanos",
telefono=123456789)
dep2 = Departamento.objects.create(nombre="Desarrollo", telefono=987654321)

# Crear habilidades
hab1 = Habilidad.objects.create(nombre="Python")
hab2 = Habilidad.objects.create(nombre="Liderazgo")
hab3 = Habilidad.objects.create(nombre="Django")

# Crear empleados
emp1 = Empleado.objects.create(nombre="Juan", fecha_nacimiento="1990-01-01",
antiguedad=5, departamento=dep2)
emp1.habilidades.add(hab1, hab3)

emp2 = Empleado.objects.create(nombre="María", fecha_nacimiento="1985-05-15",
antiguedad=10, departamento=dep1)
emp2.habilidades.add(hab2)
```

CRUD Completo

A continuación, implementaremos las operaciones de **Crear**, **Leer**, **Actualizar** y **Eliminar** para el modelo `Empleado`.

Crear (Create)

Vista para crear empleados

En `empresa/views.py`:

```python
from django.shortcuts import render, redirect
from .models import Empleado, Departamento, Habilidad

def crear_empleado(request):
    if request.method == 'POST':
        nombre = request.POST['nombre']
        fecha_nacimiento = request.POST['fecha_nacimiento']
        antiguedad = request.POST['antiguedad']
        departamento_id = request.POST['departamento']
        habilidades_ids = request.POST.getlist('habilidades')

        departamento = Departamento.objects.get(id=departamento_id)
        empleado = Empleado.objects.create(nombre=nombre,
fecha_nacimiento=fecha_nacimiento, antiguedad=antiguedad,
departamento=departamento)
        empleado.habilidades.set(habilidades_ids)
        return redirect('listar_empleados')

    departamentos = Departamento.objects.all()
    habilidades = Habilidad.objects.all()
    return render(request, 'empresa/crear_empleado.html', {'departamentos':
departamentos, 'habilidades': habilidades})
```

Template para crear empleados

Crear un archivo `templates/empresa/crear_empleado.html`:

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Crear Empleado</title>
</head>
<body>
    <h1>Crear Empleado</h1>
    <form method="POST">
        {% csrf_token %}
        <label for="nombre">Nombre:</label>
        <input type="text" id="nombre" name="nombre" required><br>

        <label for="fecha_nacimiento">Fecha de Nacimiento:</label>
        <input type="date" id="fecha_nacimiento" name="fecha_nacimiento"
required><br>

        <label for="antiguedad">Antigüedad:</label>
        <input type="number" id="antiguedad" name="antiguedad" value="0"
required><br>

        <label for="departamento">Departamento:</label>
        <select id="departamento" name="departamento">
            {% for departamento in departamentos %}
            <option value="{{ departamento.id }}">{{ departamento.nombre }}
</option>
            {% endfor %}
        </select><br>

        <label for="habilidades">Habilidades:</label><br>
        {% for habilidad in habilidades %}
        <input type="checkbox" name="habilidades" value="{{ habilidad.id }}">
{{ habilidad.nombre }}<br>
        {% endfor %}

        <button type="submit">Crear</button>
    </form>
</body>
</html>
```

URLs

Crear un fichero `empresa/urls.py`:

```
from django.urls import path
from .views import listar_empleados, crear_empleado, editar_empleado,
eliminar_empleado

urlpatterns = [
    path('empleados/crear/', crear_empleado, name='crear_empleado'),
]
```

Y desde `empresa\urls.py` incluir las rutas definidas en la aplicación:

```
from django.urls import path, include

import empresa

urlpatterns = [
    #    path('admin/', admin.site.urls),
    path('empresa/', include('empresa.urls')),
]
```

Ahora haz pruebas desde la url desde el navegador:

Y prueba a crear un empleado comprobando su inserción desde el gestor de BBDD.

Listado

Vista para listar empleados

En `empresa/views.py`:

```
def listar_empleados(request):
    empleados = Empleado.objects.all()
    return render(request, 'empresa/listar_empleados.html', {'empleados':
empleados})
```

Template para listar empleados

El código del template del listado de empleados `empresa/listar_empleados.html`:

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Lista de Empleados</title>
</head>
<body>
    <h1>Lista de Empleados</h1>
    <table border="1">
        <thead>
            <tr>
                <th>Nombre</th>
                <th>Fecha de Nacimiento</th>
                <th>Antigüedad</th>
                <th>Departamento</th>
                <th>Habilidades</th>
                <th>Acciones</th>
            </tr>
        </thead>
        <tbody>
            {% for empleado in empleados %}
            <tr>
                <td>{{ empleado.nombre }}</td>
                <td>{{ empleado.fecha_nacimiento }}</td>
                <td>{{ empleado.antiguedad }}</td>
                <td>{{ empleado.departamento.nombre }}</td>
                <td>{% for habilidad in empleado.habilidades.all %}{{
habilidad.nombre }}{% if not forloop.last %}, {% endif %}{% endfor %}</td>
                <td>
                   {% comment %} <a href="{% url 'editar_empleado' empleado.id
%}">Editar</a>
                    <a href="{% url 'eliminar_empleado' empleado.id
%}">Eliminar</a>{% endcomment %}
                </td>
            </tr>
            {% endfor %}
        </tbody>
    </table>
    <a href="{% url 'crear_empleado' %}">Crear Nuevo Empleado</a>
</body>
</html>
```

URLs

Añade la url en `empresa/urls.py`:

```
from django.urls import path
from .views import listar_empleados, crear_empleado, editar_empleado,
eliminar_empleado

urlpatterns = [
    path('empleados/', listar_empleados, name='listar_empleados'),
    path('empleados/crear/', crear_empleado, name='crear_empleado'),
]
```

Ahora haz pruebas desde la url desde el navegador.

Detalle

Vista para listar empleados

En `empresa/views.py`:

```
def ver_empleado(request, empleado_id):
    empleado = Empleado.objects.get(id=empleado_id)
    return render(request,'empresa/ver_empleado.html',{'empleado': empleado})
```

Template para ver el detalle de un empleado

El código del template del listado de empleados `empresa/ver_empleado.html`:

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Detalle de empleado</title>
</head>
<body>
    <h1>Detalle de empleado</h1>
    <table border="1">
        <thead>
            <tr>
                <th>Imagen</th>
                <th>Nombre</th>
                <th>Fecha de Nacimiento</th>
                <th>Antigüedad</th>
                <th>Departamento</th>
                <th>Habilidades</th>
            </tr>
        </thead>
        <tbody>
            <tr>
                <td>
                    {% if empleado.imagen %}
                        <img style="max-width: 10em; height: auto; object-fit:
cover;" src="{{ empleado.imagen.url }}" alt="Imagen de {{ empleado.nombre }}"
/>
                    {% else %}
                        <p>No hay imagen disponible.</p>
                    {% endif %}
                </td>
                <td>{{ empleado.nombre }}</td>
                <td>{{ empleado.fecha_nacimiento }}</td>
                <td>{{ empleado.antiguedad }}</td>
                <td>{{ empleado.departamento.nombre }}</td>
                <td>{% for habilidad in empleado.habilidades.all %}{{
habilidad.nombre }}{% if not forloop.last %}, {% endif %}{% endfor %}</td>
            </tr>
        </tbody>
    </table>
</body>
</html>
```

URLs

Añade la url en `empresa/urls.py`:

```
from django.urls import path
from .views import listar_empleados, crear_empleado, editar_empleado,
eliminar_empleado

urlpatterns = [
    ...
    path('empleados/<int:empleado_id>', ver_empleado, name='ver_empleado'),
    ...
]
```

Ahora modifica la vista del listado de empleados para que en el nombre del empleado pongamos un enlace a la vista de detalle:

```
<td><a href="{% url 'ver_empleado' empleado.id %}">{{ empleado.nombre }}</a>
</td>
```

Actualizar (Update)

Vista para editar empleados

En `empresa/views.py`:

```
def editar_empleado(request, empleado_id):
    empleado = Empleado.objects.get(id=empleado_id)

    if request.method == 'POST':
        empleado.nombre = request.POST['nombre']
        empleado.apellido = request.POST['apellido']
        departamento_id = request.POST['departamento']
        habilidades_ids = request.POST.getlist('habilidades')

        empleado.departamento = Departamento.objects.get(id=departamento_id)
        empleado.habilidades.set(habilidades_ids)
        empleado.save()
        return redirect('listar_empleados')

    departamentos = Departamento.objects.all()
    habilidades = Habilidad.objects.all()
    return render(request, 'empresa/editar_empleado.html', {
        'empleado': empleado,
        'departamentos': departamentos,
        'habilidades': habilidades,
    })
```

Template para editar empleados

Crear un archivo `templates/empresa/editar_empleado.html` similar al de crear empleados, pero con los campos prellenados:

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Editar Empleado</title>
</head>
<body>
    <h1>Editar Empleado</h1>
    <form method="POST">
        {% csrf_token %}
        <label for="nombre">Nombre:</label>
        <input type="text" id="nombre" name="nombre" value="{{ empleado.nombre
}}" required><br>

        <label for="fecha_nacimiento">Fecha de Nacimiento:</label>
        <input type="date" id="fecha_nacimiento" name="fecha_nacimiento"
value="{{ empleado.fecha_nacimiento }}" required><br>

        <label for="antiguedad">Antigüedad:</label>
        <input type="number" id="antiguedad" name="antiguedad" value="{{
empleado.antiguedad }}" required><br>

        <label for="departamento">Departamento:</label>
        <select id="departamento" name="departamento">
            {% for departamento in departamentos %}
            <option value="{{ departamento.id }}" {% if departamento.id ==
empleado.departamento.id %}selected{% endif %}>
                {{ departamento.nombre }}
            </option>
            {% endfor %}
        </select><br>

        <label for="habilidades">Habilidades:</label><br>
        {% for habilidad in habilidades %}
        <input type="checkbox" name="habilidades" value="{{ habilidad.id }}" {%
if habilidad in empleado.habilidades.all %}checked{% endif %}>
        {{ habilidad.nombre }}<br>
        {% endfor %}

        <button type="submit">Guardar Cambios</button>
    </form>
</body>
</html>
```

URLs

Añade la url en `empresa/urls.py`:

```
from django.urls import path
from .views import listar_empleados, crear_empleado, editar_empleado,
eliminar_empleado

urlpatterns = [
     path('empleados/', listar_empleados, name='listar_empleados'),
     path('empleados/crear/', crear_empleado, name='crear_empleado'),
     path('empleados/editar/<int:empleado_id>/', editar_empleado,
name='editar_empleado'),
]
```

Ahora haz pruebas desde el navegador descomentando el enlace que hay en el template `listar_empleados.html`.

Eliminar (Delete)

Vista para eliminar empleados

En `empresa/views.py`:

```
def eliminar_empleado(request, empleado_id):
    empleado = Empleado.objects.get(id=empleado_id)
    empleado.delete()
    return redirect('listar_empleados')
```

URLs

En `empresa/urls.py`:

```
from django.urls import path
from .views import listar_empleados, crear_empleado, editar_empleado,
eliminar_empleado

urlpatterns = [
    path('empleados/', listar_empleados, name='listar_empleados'),
    path('empleados/crear/', crear_empleado, name='crear_empleado'),
    path('empleados/editar/<int:empleado_id>/', editar_empleado,
name='editar_empleado'),
    path('empleados/eliminar/<int:empleado_id>/', eliminar_empleado,
name='eliminar_empleado'),
]
```

Ahora haz pruebas desde el navegador descomentando el enlace que hay en el template `listar_empleados.html`.